

1 Introduction

Augmented reality (AR) systems rely on the ability to accurately estimate the pose of a physical object or surface from camera imagery and then render virtual content into the camera view in real time. While modern AR platforms increasingly depend on specialized hardware and sophisticated feature-tracking pipelines, there remains substantial value in developing lightweight, easily deployable systems that illustrate AR fundamentals using commodity sensors and open-source software. Such systems provide an interpretable link between the computer vision processes used to interpret visual information and the graphics processes used to synthesize virtual scenes, making them ideal for educational settings, rapid prototyping, and research demonstrations.

The central objective is formulated through the following problem statement:

How can a monocular AR system recover camera pose in real time from a printed planar marker and use this pose to render a virtual object with approximately correct 3D position and orientation in the camera view?

Marker-based AR is useful in this project because it makes the core underlying visual computing pipeline explicit: The camera model is described by estimated intrinsic parameters, marker pose is computed from identifiable visual features, and rendering follows from a transformation pipeline in 3D space. Marker-based AR therefore serves as a clear and accessible demonstration of how computer vision and computer graphics can be integrated to create a perceptual illusion of co-located real and virtual content. Additionally, investigating the performance limitations of such a system including jitter, sliding, degraded tracking at steep angles, and sensitivity to illumination, provides insight into the practical constraints of real-world AR implementations.

In this project, a printed ArUco marker is used as the reference target. OpenCV is employed to detect marker corners and perform pose estimation via `estimatePoseSingleMarkers()`, while simple virtual geometry (a cube) is rendered using OpenGL. The system is evaluated across multiple environmental and imaging conditions, including static pose stability, lighting variations, and changes in camera viewing angle. Experimental results are logged automatically and analyzed to characterize system latency, pose noise, and visual alignment. The outcome is a functional prototype that demonstrates both the capabilities and limitations of low-cost marker-based AR, and highlights potential directions for improvement such as smoothing filters, multi-marker layouts, or alternative tracking strategies.

2 Method

The system follows a marker-based monocular AR pipeline. A printed 6x6 ArUco marker is used as a planar reference target, while a consumer-grade webcam provides the live image input. Before pose estimation, the camera is calibrated using a printed chessboard pattern. The resulting calibration file contains the intrinsic camera matrix and distortion coefficients, which describe the mathematical camera model used by OpenCV.

For each video frame, OpenCV detects the ArUco marker and extracts its 2D corner positions. Since the physical marker size is known, these 2D image points can be related to known 3D marker coordinates. An extrinsic pose estimation routine is then used to estimate the marker pose relative to the camera, represented as a rotation vector and translation vector.

To render a virtual object at the estimated pose, the rotation vector is converted into a rotation matrix using the Rodrigues formula. This rotation and the corresponding translation are combined into a 4x4 transformation matrix, which is used as the model matrix for the virtual cube. The final output is produced by rendering the webcam image as a background texture and drawing the cube on top using OpenGL.

3 Implementation

The prototype was implemented in C++ using OpenCV for image processing and pose estimation, and OpenGL for rendering. Frames are captured at 1280x720 resolution using a Logitech C270 webcam via `cv::VideoCapture`. A short camera warm-up step is performed before rendering begins to reduce unstable initial frames.

Marker detection is performed every frame using `cv::aruco::detectMarkers()` with a 6x6 dictionary. Subpixel corner refinement and adaptive threshold parameters were enabled to improve detection stability under typical indoor lighting. If no marker is detected, the cube is hidden for that frame.

The camera calibration data is loaded from a YAML file at startup. This allows pose estimation to use measured camera intrinsics rather than a rough default camera model. Once a marker is detected, `estimatePoseSingleMarkers()` returns rotation and translation vectors. These are converted into a model matrix and passed to the cube renderer. The cube is drawn with a dedicated shader that receives model, view, and projection matrices. Depth testing is enabled so the cube faces render correctly during rotation.

The webcam frame is uploaded to GPU memory as a texture and rendered onto a screen-aligned quad. The virtual cube is then rendered on top of the video stream. During development, practical issues such as image flipping, coordinate conventions, and visible pose jitter affected alignment. These were handled through frame orientation adjustments and experimental tuning of marker size, cube scale, and detector parameters.

For evaluation, pose values are logged to CSV files whenever a marker is detected. The logged translation and rotation values are later used to analyze pose stability, lighting robustness, and behavior under different viewing angles.

4 Experiments and Analysis

To evaluate the performance and robustness of the marker-based AR system, a set of controlled experiments was conducted focusing on pose stability, lighting sensitivity, and viewing angle variation. All experiments were executed using a Logitech C270 webcam operating at 1280x720 resolution, with pose estimated via OpenCV's `estimatePoseSingleMarkers()` and a 6x6 ArUco marker printed on a piece of A4 paper. For each test, pose values were recorded to a log file using the integrated CSV output module.

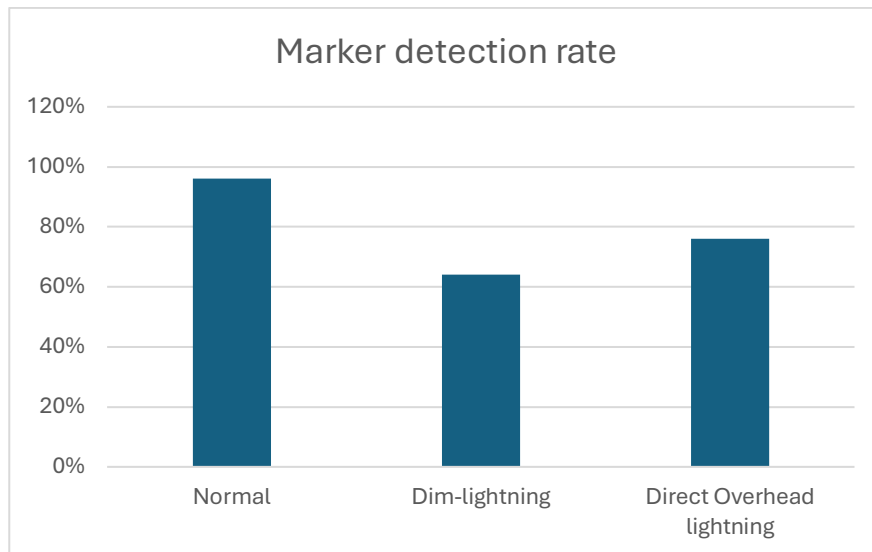
4.1 Pose Stability Under Static Conditions

The first experiment assessed baseline stability when both the camera and marker remained static. The webcam was positioned at a fixed distance of approximately 30 cm from the marker, and a 10-second video sequence was recorded, corresponding to roughly 300 frames of pose data.

To quantify jitter, the standard deviation of the estimated translation components tx , ty , tz and rotation components rx , ry , rz was computed. Under these conditions, tx and ty fluctuated only minimally around their mean values, on the order of 0.002 meters. Even when the physical setup was completely static, the rendered cube exhibited visible micro-movement on screen. While the jitter did not prevent marker detection, it produced minor floating behavior in the cube.

4.2 Lighting Robustness

A second experiment examined the sensitivity of marker detection and pose estimation to different illumination conditions. Three lighting scenarios were tested: ambient indoor lighting, dim lighting with only illumination from the computer screens, and direct overhead lighting from a powerful desk lamp positioned right above the marker. For each lighting configuration, a 10-second recording was captured and detection rate was calculated as the proportion of frames in which the marker was successfully detected.



Under normal lighting, detection performance was stable, with the marker being detected in nearly all frames. The most significant degradation occurred under dim lighting, where the detection rate decreased noticeably, with intermittent frame losses resulting in a detection rate of only 64%. Under direct overhead illumination, highlights and glare were visible on the marker surface, causing the detection rate to drop to 76%. Glare also increased pose noise, particularly in the depth dimension tz , leading to larger visible fluctuations in cube position.

These results indicate that lighting variation remains a key challenge in marker-based AR. Although detection is generally robust in diffuse light, specular highlights and low-contrast conditions adversely affect corner identification, ultimately reducing pose stability and producing noticeable sliding or “float” in the rendered cube.

4.3 Variation in Viewing Angle

The third experiment evaluated tracking performance when the camera was rotated relative to the marker plane. The webcam was gradually tilted away from a frontal view to increasingly steep angles while continuously logging pose data. While the system maintained detection across a broad range of angles, pose accuracy deteriorated visibly once the marker was viewed at steep angles. This manifested as an apparent sliding of the cube across the marker surface, particularly along the plane of rotation.

The cube tended to “swim” on the marker when the camera was tilted past this threshold. The behavior is attributable to reduced corner localization precision when fewer inner marker edges remain visible, and the perspective distortion becomes extreme.

The observed sliding effect reflects a known limitation of single-marker pose estimation, which relies on accurate corner positions in pixel space. In practical settings where wide camera movement is expected, multi-marker layouts or temporal filtering would be recommended to maintain a better spatial coherence.

4.4 Performance and Latency Observations

System performance was evaluated through real-time frame rate measurements during each experimental condition. Across all trials, the system maintained an average frame rate of approximately 24 FPS on the GPU pipeline, with occasional dips to 13 FPS under dim or high-glare lighting. These reductions corresponded directly to increased per-frame pose estimation time and reduced marker detection frequency.

Although overall latency remained low enough to permit interactive visualization, fluctuations in frame rate correlated with increased pose noise. Higher processing time resulted in proportionally fewer stable continuous detections and greater visible jitter in the rendered object. This observation supports the hypothesis that latency and pose stability are coupled in lightweight AR systems, particularly when computation and rendering share a single CPU/GPU pipeline. Occasional freezes were

6 Conclusion

This project demonstrates that marker-based augmented reality can be achieved with modest hardware and open-source software. Although the pose estimate is not perfectly stable, the system reliably detects markers and renders a virtual object in approximate alignment with the physical scene, albeit with a y-axis offset. Latency remained low enough for interactive use, and calibration improved consistency compared to default camera intrinsics.

The approach is particularly relevant in educational contexts because it makes the AR pipeline visible and understandable. Instead of treating AR as a black-box application, students can see how a printed marker is detected, how its pose is estimated, and how this pose is transformed into a rendered object. In a school environment, printed markers could be attached to cards representing animals, geometric solids, or planets. Because the setup requires only a laptop, a webcam, and printed markers, it is both affordable and portable.

Future work could improve the system through temporal smoothing, multi-marker layouts, better calibration validation, or interactive behavior based on marker identity. For example, different markers could trigger different virtual animals or objects, making the prototype more suitable for classroom demonstrations. Overall, the project shows that even a simple marker-based AR system can be a useful tool for visual learning, rapid prototyping, and understanding the connection between computer vision and computer graphics.